



Internship report (2A ENSEA)

Deep-based generation of Wing Interferential Patterns Images for
the surveillance of blood-sucking insect population by Machine
learning algorithms

Jorge Andrés HERNÁNDEZ GALEANO

Internship carried out from 15/05/2023 to 15/08/2023

Internship tutors : Mme. Camille SIMON-CHANE, M. Aymeric HISTACE

Referent professor : M. Ayoub JEBRI

Educational institution : École Nationale Supérieure de l'Électronique et de ses Applications (ENSEA)

Internship host research laboratory : Equipes Traitement de l'Information et Systèmes (ETIS Lab)

6 avenue du Ponceau, 95014 Cergy-Pontoise cedex



Acknowledgments

I would like to express my sincere gratitude to my two mentors Camille Simon Chane and Aymeric Histace, whose guidance and support have been vital throughout the duration of my internship. Their invaluable insights, patience, and encouragement allowed me to delve into the project with enthusiasm and dedication. This internship experience has been incredibly rewarding, largely due to the opportunities they provided me to freely explore, make decisions and develop the project under their expert mentorship. I am truly thankful for their mentorship, the opportunity they have provided me, and the knowledge I have gained through this experience.

Abstract

This internship report addresses the task of generating images of wing interference patterns for dataset augmentation within the context of monitoring and accurately identifying diptera specimens to prevent disease spread. These efforts play a crucial role in targeted vaccination campaigns and disease elimination. Current identification methods rely on experts and expensive techniques. The recent work of the ETIS Lab focuses on precise diptera identification using deep learning architectures and Wing Interference Patterns (WIPs), thus mitigating the reliance on these methods. However, challenges involving imbalanced and underrepresented specimens impede the recognition of certain species. This work aims to explore generative architectures that have the potential to enhance the WIPs' dataset. The objectives encompass gaining a comprehensive understanding of GANs, analyzing the current WIPs dataset, implementing and testing GAN architectures tailored to the dataset, augmenting eight underrepresented classes belonging to the *Glossina* genus, and evaluating these synthetic images. As a result, the generated images exhibit a visual perceptual resemblance, although some lack texture details and exhibit a blurry appearance, particularly in images with a low number of representatives in the original dataset. T-SNE analysis indicates that the features of the generated images are not entirely within the same space as the original images; conversely, distinct clusters for both original and generated images are formed..

Key words: Wing Interference Patterns, dataset augmentation, diptera specimens, generative adversarial networks (GANs), image generation, T-SNE analysis.

Contents

1	Introduction	1
2	Work environment	2
2.1	The host Laboratory	2
2.2	The host team	2
3	Internship description	4
3.1	Context	4
3.2	Objectives	5
4	Literature review	6
4.1	Generative adversarial Networks (GANs)	6
4.1.1	Training of GANs	8
4.2	Challenges and Problems of GANs	8
4.2.1	Vanishing Gradients	8
4.2.2	Mode Collapse	9
4.2.3	Failure to Converge	9
4.3	Architectures	9
4.3.1	Adversarial Auto-encoder	9
4.3.2	Convolutional Network	10
4.3.3	Residual Network	10
5	Methodology	12
5.1	Tools employed	12
5.1.1	Hardware	12
5.1.2	Software	12
5.2	Dataset Analysis	12
5.2.1	Data cleaning and loading	12
5.2.2	WIPs dataset	13
5.3	Architectures exploration and optimization	14
5.3.1	Adversarial Autoencoder	15
5.3.2	DCGAN	15
5.3.3	GAN with ResNet architecture	16
6	Results and discussion	19
6.1	Network Evaluation and Selection	19
6.2	Image Evaluation	19
6.3	Dimensional Reduction	24
7	Conclusion and perspectives	27

1 Introduction

Wing Interferential Patterns (WIPs) are color patterns exhibited by wings with a thin membrane in dark environment when exposed to incoming external light, these patterns are distinctive for different diptera species and suggest potential avenue for taxonomic identification [1]. Classification and adequate surveillance of these specimens is of vital importance in curbing the propagation of well-known and identified diseases attributed to them, such as the Human African Trypanosomiasis (HAT) caused by *Glossina specimens*. Following this aim, significant efforts to improve the current identification techniques have been taken from the ETIS Lab by using WIPs for automated classification of Diptera specimens. Methods such as this requires a large amount of images for accurate training. However, the acquisition of this images is time-consuming and requires detailed care to preserve, prepare, collect and pre-process the samples.

During this internship, we delve into a method that goes beyond classical data augmentation strategies by leveraging Generative Adversarial Networks. The goal is to generate new WIP images that would allow to fortify the dataset used for the automated classification of these specimens. The primary contribution of this effort lies in the investigation and adaptation of available generative architectures to expand the counts of WIPs samples across eight different *Glossina* specimens. It seeks to enhance our understanding of generative techniques and their potential impact on augmenting the WIPs datasets for improved automated diptera classification.

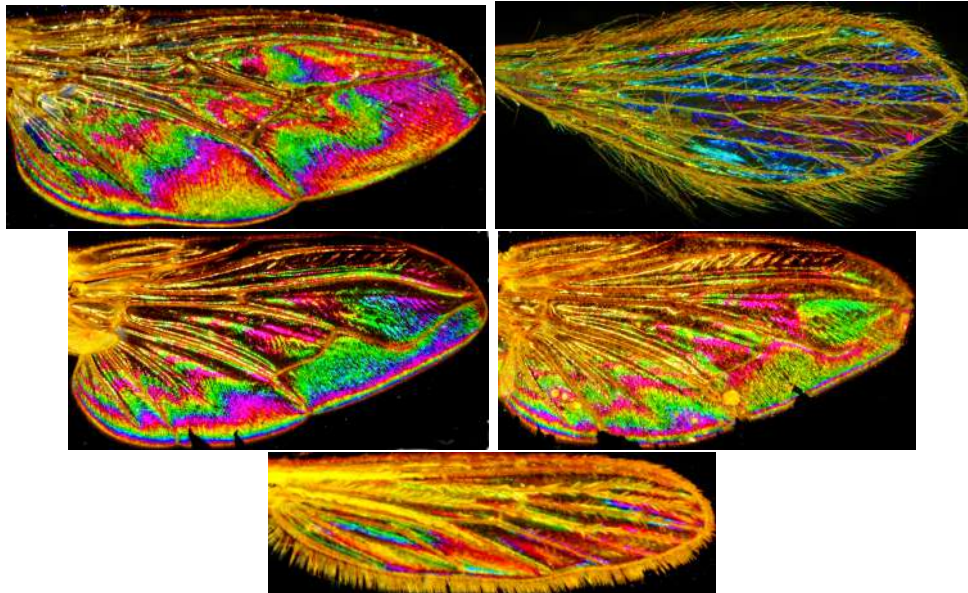


Fig. 1: Example of WIPs for five different genera

2 Work environment

2.1 The host Laboratory

The present internship was conducted at the ETIS Laboratory, a collaborative research department of CYU Cergy Paris University, ENSEA and CNRS/INS21. This laboratory is focused on theoretical and experimental research in the fields of information theory and processing systems. ETIS is up today composed of 154 members from UCP, ENSEA and CNRS including professors, CNRS researchers, engineers, technicians and PhD students.

ETIS is the primary reference in its related fields in Cergy and the north-west of Île-de-France, playing a prominent role in the cutting-edge learning with a low footprint. The laboratory is dedicated to directing its endeavors across various domains, spanning from fundamental concerns to practical applications, in response to challenges posed by societal transitions. Currently, ETIS is actively engaged in and at the forefront of the ongoing AI revolution.^[2]

The lab is actively promoting and supporting Equality-Parity actions through support of diversity acceptance and a collaboration spirit and respect among its members. Additionally, it takes proactive steps to support women in science, aiming to raise awareness about gender parity issues, encourage female students to pursue STEM domains, provide company and support for female researchers after maternity leave, and combat instances of sexual harassment..



2.2 The host team

The hosting team for the internship was the CELL team at the head of the professor **Aymeric Histace**. The CELL team operates in the realm of Electronics, Signal and Image Processing for the Modeling and Design of Reconfigurable, Communicating, Reliable and Intelligent Embedded Systems ^[3].



The work conducted during this period was undertaken as part of the CELL team's ongoing Deeptera project, under the guidance of the professor **Camille Simon Chane**. This project centers on embedded IA for automatic classification of bloodsucking insects using a particular imaging modality called WIPS.

3 Internship description

3.1 Context

Monitoring and accurate identification of diptera population in the field is a crucial challenge aimed at preventing the spread of unforeseen diseases. It also aids in focusing vaccination campaigns within population in order to contribute to the future elimination of high impact diseases like the Human African Trypanosomiasis (HAT) caused by the specimens of the genus *Glossina*. Most of the current identification methods rely in few well-trained experts and expensive, destructive and time-consuming techniques like DNA barcoding and mass spectrometry protein profiling. Recently, significant efforts have been taken from the ETIS Lab for the accurate identification of these specimens using new methodologies involving optimized deep learning architectures and the use of non-destructive methods employing a new imaging type called Wing Interference Patterns(WIPs).

Some of these undertaken efforts are encountered in [4] and [1]. In [4] the authors proposed the use of WIPs in conjunction with three low computationally cost optimized deep learning architectures for their use in the surveillance of *Hematophagous Diptera* specimens using mobile applications, they achieved astonishing results in their precision tests (over 95%) executing genus classification, species classification knowing the genus, and direct species classification. Likewise, in [1] the authors investigated and illustrated that utilizing WIPs is an excellent option for the automatized identification of distinct *Glossina* species, yielding remarkably precise outcomes. Nevertheless, the authors acknowledged encountering imbalanced classes where, for some species, they had very few pictures of WIPs, and consequently, the algorithm failed at correctly identifying them. In response to this challenge and with the aim of improving their results, the authors suggested as a future development to enhance the dataset by improving species diversity and increasing the number of samples in specimens with a low number of representatives by using GANs, a kind of generative neural networks which would allow to generate new samples that resemble the original training dataset.

3.2 Objectives

The work of this internship follow the propounded future work of [4] and [1]. Specifically, the primary objective is to conduct a first exploration into generative architectures. These architectures are aimed to enhance robustness and variability of the existing WIPS dataset, and thereby tackling the challenges of species missclassification found in the previous research efforts.

To acomplish this main purpose, several key objectives have been delineated:

- Understand GAN fundamentals: Develop a deep understanding of Generative Adversarial Networks(GAN) and their components.
 - Task: Study of GAN architectures, training procedures, available tools, loss functions and its variations in order establish a strong groundwork to be applied to the WIPs dataset.
- Dataset Analysis and preprocessing: Analyze the current WIPs dataset identifying class imbalances and images quality.
 - Task: Perform the necessary preprocessing to ensure a clean and well-structured dataset, and inspect the class distributions
- GAN implementation and Training: Develop and implement different GAN architectures customized for generating synthetic WIPs images.
 - Task: Design the Generator and Discriminator networks selecting the appropriate loss and fine-tuned parameters adjusted to the working dataset.
- Evaluate synthetic Images: Assess the quality and diversity of the generated images by the GAN architectures.
 - Task: Evaluate qualitatively and quantitatively the generated images with respect to the training WIPs dataset. Use of metrics such as Fréchet Inception Distance (FID) to test the similarity between real and generated distributions.
- Augmenting Underrepresented Classes: Use the explored trained GAN to augment the dataset, focusing on classes with a low number of species.

4 Literature review

4.1 Generative adversarial Networks (GANs)

A GAN is a type of model which combines a set of modules, typically a discriminator and a generator which try to compete each other, to generate new synthetic data similar to the original training dataset. GANs were first introduced by Ian Goodfellow and colleagues in 2014 in the paper named "Generative adversarial Nets" [5], where the general architecture, training procedure and theoretical results about convergence and optimality were exposed.

GANs were developed to address the problems of other deep generative models, such as those modeled by Markov chain Monte Carlo(MCMC) based algorithms and Variational Autoencoders (VAEs). These models exhibit among other things, the difficulty of computational slowness, inaccuracy and intractable probabilistic computations [6].

These newly proposed models have garnered significant attention in recent years, and their development has evolved in an astonishing manner, consistently achieving surprising results in their capabilities year after year. One of the reasons of its research interest is its characteristic capability that allows the reconstruction of natural image manifolds producing more convincing solutions than the classical MSE-solution, see Figure 2. GANs applicability is growing in many fields and have worked well in several realistic tasks such as image generation, video generation, domain adaptation and image super-resolution [6].

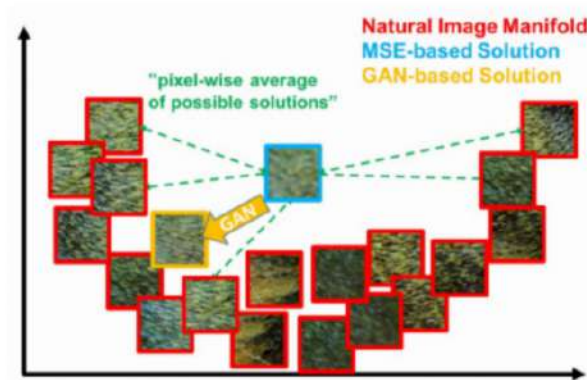


Fig. 2: Illustration of patches from the natural image manifold (red) and super-resolved patches obtained with MSE (blue) and GAN (orange). Image from [7]

The basic and general architecture of a GAN is shown in Figure 3

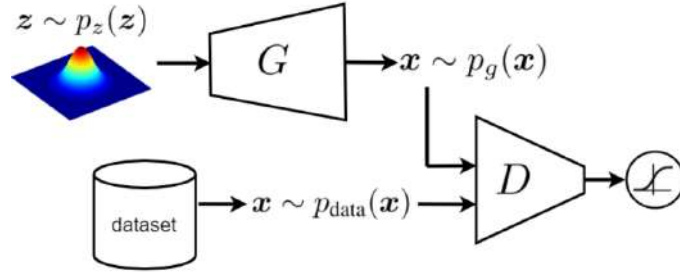


Fig. 3: Basic structure of a GAN [8]

This network consists of a generator(G) and a discriminator(D). The **generator** generates synthetic data based on a random input, typically called noise, and extracted from a Gaussian distribution. Then the generated data is compared with the real data by the **discriminator**, which in its basic form assesses the probability of the data being real. Here is when the minimax game come out: The generator tries to generate data as real as possible to fool the discriminator and the discriminator tries to counteract it by better classifying the data. This is expressed by the value function $V(G, D)$:

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)}(\log D(x)) + \mathbb{E}_{z \sim p(z)}(\log(1 - D(G(z)))) \quad (1)$$

This equation is typically used during the training procedure as a loss function. The first term of Equation 1 is constant for the generator and only used by the discriminator, it aims to output 1, the largest label, in D when the input is the real data. In the second term of Equation 1, the discriminator aims to assign the smallest label to the input data, since this is the generated one, but at the same time the generator aims to minimize it by reaching the largest label in D .

In this context, the optimal solution of a GAN is reached when the distribution of the generated data equals the distribution of the real data and the discriminator classifies the generated data with 50% of probability of being real. More details about the corollary and deduction of these results are found in [8].

In the vanilla GAN proposed in [5], the generator and discriminator were conceived as 2 independent multi-layer perceptron neural networks trained using stochastic gradient descent. However, since then different versions has been proposed using blocks of convolutional neural networks, more complex architectures and optimization algorithms. It is then possible to say that the main GANs design and optimization components are as described in [6]:

- Network Architecture
- Loss Function
- Optimization Algorithm

4.1.1 Training of GANs

Different variations of training may be needed depending on the architecture and specific loss. However, in general, the training procedure can be broadly summarized by the following algorithm:

Algorithm 1 GAN's Training Procedure . n: number of training iterations. k: number of training steps for the discriminator. m: batch size. G: generator network

```

for n iterations do
  for k steps do
     $x \leftarrow$  sample a batch of real data,  $x \sim p_{data}$ 
     $z \leftarrow$  noise samples from noise prior,  $z \sim p_g$ 
     $\hat{x} \leftarrow G(z)$ 
     $L_D \leftarrow Loss_D(x, \hat{x})$  Loss computation for discriminator
     $Optimizer(L_D)$  Gradients calculation and update of discriminator parameters
  end for
   $z \leftarrow$  noise samples from noise prior,  $z \sim p_g$ 
   $L_G \leftarrow Loss_G(G(z))$ . Loss Computation for generator
   $Optimizer(L_G)$  Gradients calculation and update of generator parameters
end for

```

While the general loss function for a GAN is represented by an equation as Equation 1, in practical terms, due to the adversarial interplay between two independent modules, this equation can be deconstructed into two distinct yet correlated equations: one governing the training of the generator and the other governing the training of the discriminator. For the case of the vanilla GAN, these are the practical equations for a batch of size m:

$$L_D = \frac{1}{m} \sum_{i=1}^m [\log(x^{(i)}) + \log(1 - D(G(z^{(i)})))] \quad (2)$$

$$L_G = \frac{1}{m} \sum_{i=1}^m [\log(1 - D(G(z^{(i)})))] \quad (3)$$

4.2 Challenges and Problems of GANs

Implementing and training GANs is quite challenging due to its high sensibility to hyper-parameters and the structure of the training data. Below, some of the most common problems that could be experienced when working with GANs are described.

4.2.1 Vanishing Gradients

It happens when the gradients of some of the network diminish close to zero or small values. It usually happens when the discriminator is too good at classifying the data and do not provide enough information for the generator to make progress. Some documented solutions include the modification of the Loss function [9].

4.2.2 Mode Collapse

One of the most common encountered problems is mode collapse, also known as Helvetica Scenario. This happens when the generator fails to generate diverse data and learns to map different points from the latent space to the same generated data point or partially very similar. This usually happens when the distribution of the training data has multiple modes that are difficult to reach for the generator[8]. Several solutions to this problem have been documented, between those can be found the use of Wassertein Losses, which reduces the distance between 2 probability distributions. Also, unrrolling which involves updating the gradients of the generator k steps after the discriminator, and finally packing that proposes training the discriminator with several packed samples of the same class at once [10].

4.2.3 Failure to Converge

It occurs in the late stages of training when the discriminator feedback is not meaningful and the generator begins to learn from unhelpful inputs. As a consequence, its own performance deteriorates. Solutions include penalizing the discriminator weights and adding noise to the inputs of the discriminator[9].

4.3 Architectures

4.3.1 Adversarial Auto-encoder

The architecture of adversarial Auto-Encoders is described in [11]. This network is composed of an autoencoder (encoder-decoder) that compresses the input image into a latent space, and then it is recovered from this by the decoder. At the same time an additional network acts as a classifier by distinguishing if the samples come from a fixed sampled distribution or from the 'actual' latent space. This results in the encoder acquiring the ability to transform the data distribution into the prior distribution, and simultaneously, the decoder learns a complex generative model that connects the imposed prior distribution to the data distribution.

It typically involves a reconstruction loss for the auto-encoder, and an adversarial loss for the encoder and classifier network.

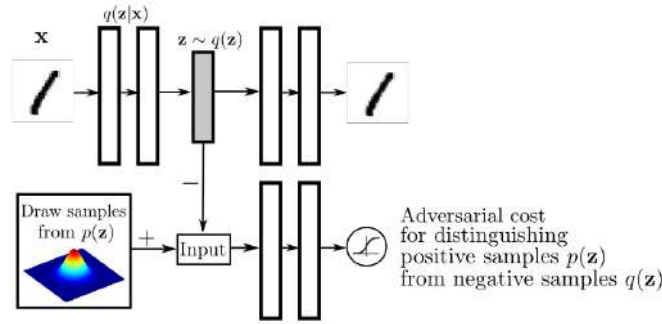


Fig. 4: Architecture of an Adversarial Autoencoder. Image from [11]

4.3.2 Convolutional Network

Use of convolutional network for GANs was first described and proposed as stable setting in [12]. This architecture was called as the Deep Convolutional GANs(DCGAN) and consist of a stack of convolutional layers that replace the multi-layer perceptron of the vanilla GAN by downsampling in the discriminator net and upsampling in the generator. This network incorporates batch normalization after each convolution layer and the use of LeakyReLU to alleviate gradient vanishing. Specific details in terms training, filters and layer interconnections are described in [12].

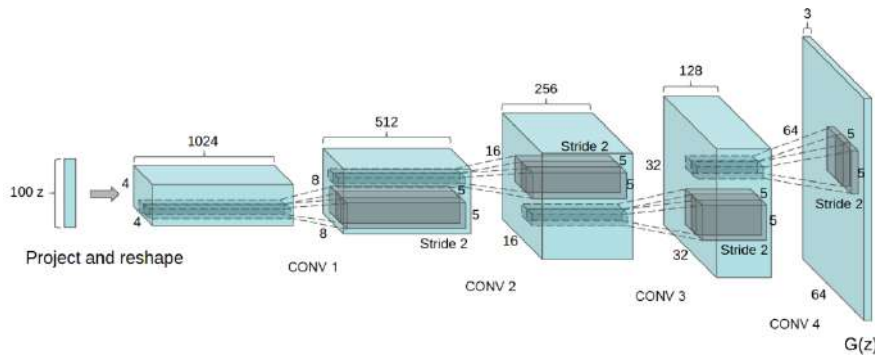


Fig. 5: DCGAN generator architecture. Image From [12]

4.3.3 Residual Network

The Residual Network architecture in GANs follows the same principle of down-sampling in the discriminator and up-sampling in the generator, however, it makes use of skipping connections. This characteristic facilitates the design of deeper networks that capture deeper features while reducing the problem of vanishing gradients and loss of information between layers. Adapted implementation of this architecture for GANs is proposed in [13], where authors achieved significantly better results than other implementations like DCGAN using Wassertein loss with gradient penalty and training in CIFAR-10, which is a famous dataset containing 32x32 images of 10 different classes including cars, birds, planes, cats, etc.

ResNet GANs are composed of Residual blocks that up-sample, down-sample or re-sample. These blocks have a specific architecture that usually follows the batch normalization, activation, and weighting operation. Figure 6 shows a typical architecture of a residual block in a GAN.

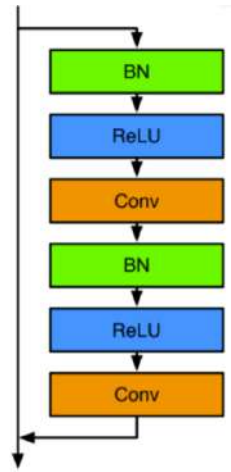


Fig. 6: Residual Block architecture. Image from [14]

	RGB image $x \in \mathbb{R}^{32 \times 32 \times 3}$
$z \in \mathbb{R}^{128} \sim \mathcal{N}(0, I)$	ResBlock down 128
dense, $4 \times 4 \times 256$	ResBlock down 128
ResBlock up 256	ResBlock 128
ResBlock up 256	ResBlock 128
ResBlock up 256	ReLU
BN, ReLU, 3×3 conv, 3 Tanh	Global sum pooling
(a) Generator	dense $\rightarrow 1$
	(b) Discriminator

Fig. 7: Example of GAN ResNet Architecture used for authors of [14]

5 Methodology

5.1 Tools employed

5.1.1 Hardware

The scripts were developed and executed on a laptop running Windows 10 Pro with 16 GB of RAM, CPU Intel Core i7-12800H and GPU Nvidia RTX A1000 12GB. Most of the training scripts were executed on the GPUs of the server cetautomatix of the Etis Lab through remote connection. This server allows the use of 2 GPU Quadro RTX 8000 46GB.

5.1.2 Software

All coding tasks were developed with Python 3.10.6 and the Open Source machine learning framework Pytorch 2.0.1+cu118.

The research framework Torchgan [15] was employed for the efficient development and training of GANs.

Likewise, the Torchmetrics library, skimage, and the scikit-learn implementation of the t-SNE algorithm were employed to assess the quality and diversity of the generated images.

5.2 Dataset Analysis

5.2.1 Data cleaning and loading

At the beginning a set of WIPs images was available with the corresponding excel document where the labeling and identification of the specimens were annotated. In order to be able to work with this data and encapsulate it in a dataset of the Pytorch framework, several cleaning and pre-processing steps were performed:

- Several inconsistencies that were found among the labeling were corrected. Between those inconsistencies, there were various typo errors in the naming of the images, non-existent images for some labels, nonuniform color spaces (there were some images with an extra alpha channel RGBA and other RGB)
- A new clean labeling csv document was generated that only contains the essential information for training, this document was used as the referent for the loading of the dataset and was named as "wips_reference.csv". This essential information includes the image's name and its corresponding category. The category label encompasses the specimens within the subspecies range
- The data was encapsulated within a dataset structure using the PyTorch framework. This was done to simplify the process of loading the data and applying image transformations.

5.2.2 WIPs dataset

The dataset is constituted of 5516 WIPs pictures of around 1500x500 pixels of which 1766 are Glossina specimens. Table 1 shows the distribution of the number of WIPs pictures for the Glossina specimens. It can be observed that the dataset is highly imbalanced and, 18 out of 23 of these specimens have less than 100 pictures and 13 have more than 15 representatives.

To enhance the dataset through augmentation, the primary focus of this effort will be on generating new images for 8 specific specimens within the Glossina Genus. These particular specimens are characterized by having a population size in the dataset ranging from 15 to 100 individuals.

Tab. 1: Glossina specimens in WIPs dataset

	Category Label	Genus	Species	Subspecies	Count
1	13	Glossina	palpalis	palpalis	619
2	10	Glossina	fuscipes	fuscipes	301
3	12	Glossina	palpalis	gambiense	191
4	11	Glossina	morsitans	morsitans	146
5	58	Glossina	Pallidipes		117
6	15	Glossina	fuscipes	quazensis	96
7	14	Glossina	tachinoïdes		94
8	46	Glossina	morsitans	submorsitans	30
9	44	Glossina	longipalpis	longipalpis	30
10	50	Glossina	Caliginea		23
11	57	Glossina	Pallicera		22
12	54	Glossina	morsitans	Centralis	22
13	51	Glossina	fusca	fusca	20
14	49	Glossina	brevipalpis		13
15	56	Glossina	nigrofusca		12
16	53	Glossina	medicorum		12
17	138	Glossina	longipennis		4
18	45	Glossina	austeni	austeni	4
19	47	Glossina	swynnertoni		3
20	139	Glossina	Vanhoofi		2
21	55	Glossina	nashi		2
22	59	Glossina	tabaniformis		2
23	52	Glossina	haningtoni		1
				Total	1766

5.3 Architectures exploration and optimization

Several architectures and model optimizations were explored following the advances in the literature and the GAN problems encountered. Below, you will find a selection of these implementations and optimizations presented in chronological order. All GAN architectures and training procedures found in the papers and example code were adapted to work properly with the Torchgan research framework [15].

For terms of efficiency, all training images were converted to tensor with values ranging between 0-1 and resized to 256x116 pixels, which is the size of resizing for the classifier used in [1]. Moreover, in order to ease and be able to work with more general models of GANs, images were padded to reach a squared size of 256x256 pixels.

Likewise, for terms of training reproducibility, a manual seed of value 999 was set for all training cases. Adam optimizer was used in all cases with learning rate 0.0001 for the generator and 0.0003 for the discriminator, betas were adjusted to (0.5, 0.999). In some cases, for the ResNet model, the learning rate was slightly tuned to 0.0002 depending on the training behavior. All detailed implementation of the models and training scripts can be found in: <https://github.com/jhernandezga/Deep-based-generation-of-WIPS>

The initial architecture explorations was done by training on the specimens of "*Sergentomyia schwetzi*", Figure 8, which comprised 100 WIPs samples. These specimens showcased a comparatively intricate texture compared to other specimens.

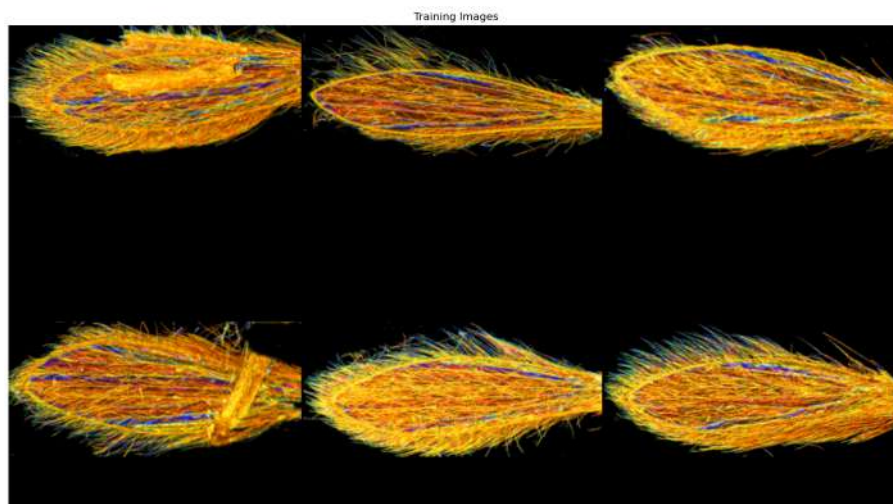


Fig. 8: *Sergentomyia schwetzi* training samples

Even though the vanilla implementation of GAN was explored using the built-in implementation of Torchgan, this architecture was discarded due to several problems of instability during training and its already registered low performance in comparison to more recent architectures.

5.3.1 Adversarial Autoencoder

An architecture of an adversarial auto-encoder was implemented in its adapted version to work on images of 256x256 pixels.

Different models were trained using, at first, the original adversarial loss and perceptual loss, and then some optimizations were attempted using the Mean squared error loss (MSE), the perceptual loss using the pre-trained VGG16 as a feature extractor, and a combination of the perceptual and Wassertein losses.

In all cases, training was stopped early after it was observed that around 2000 epochs, the training had seemingly reached a plateau. No substantial improvement in the generated images was observed at this point. The images were simply qualitatively assessed since they did not exhibit the expected result. Some of the generated samples reached by this architecture are shown in images 9,10 and 11.

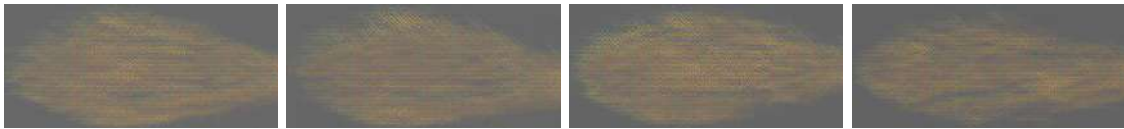


Fig. 9: Generated images AEE perceptual loss

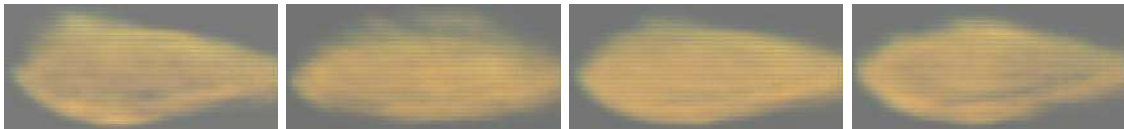


Fig. 10: Generated images AEE perceptual-Wassertein loss

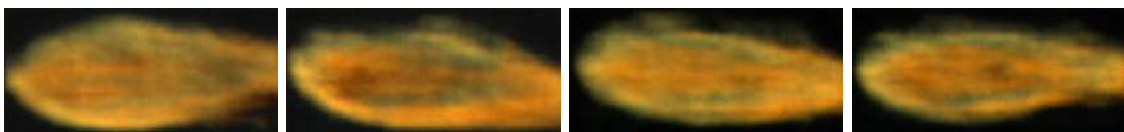


Fig. 11: Generated images AEE MSE loss

5.3.2 DCGAN

Employing a simpler and standard model, an architecture was developed involving sampling and up-sampling through convolutions and transposed convolutions. For this purpose, the built-in Torchgan implementation of DCGAN was utilized in combination with various loss functions, including the adversarial, least squares losses and finally the Wassertein loss with gradient penalty. Among these losses, the only one that reached stable training and was robust enough to generate WIPs images was the Wassertein loss with gradient penalty using slightly different learning rates for the generator and discriminator(0.0001 and 0.0003)

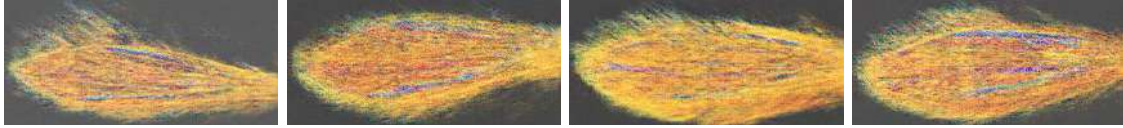


Fig. 12: Generated images DCGAN Wassertein-GP loss

Taking advantage of the built-in network available in the Torchgan framework, an adapted Boundary Equilibrium GAN [16] was trained for approximately 2000 epoch. Results of this network are shown in Figure 13

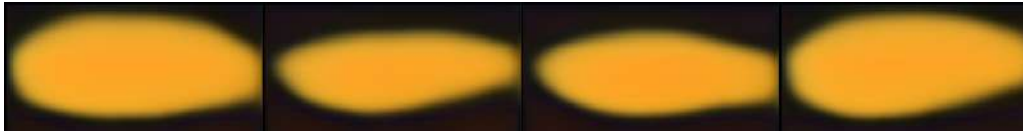


Fig. 13: Generated Images BEGAN

In this case, the network only recovers the shape and global color of the training images and does not surpass the result already reached by the DCGAN architecture.

5.3.3 GAN with ResNet architecture

Seeking to enhance and optimize the architecture, an implementation using residual blocks was employed. As documented in Gulrajani et al. [13], the integration of residual blocks with the Wasserstein GAN framework, along with gradient penalty, notably enhances the quality of the generated images.

As observed in Figure 14 the quality of the generated images improved significantly, recovering at the same time more details of the texture and color of the wings. However, an evident lack of diversity in the images is observed

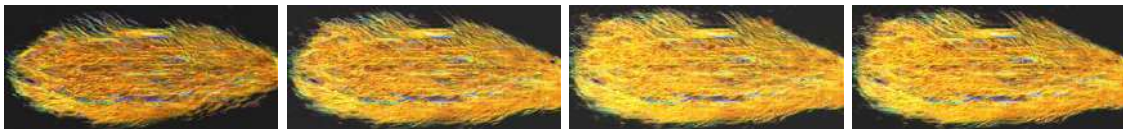


Fig. 14: Generated Images WGAN-GP ResNet

In order to solve this problem, some modifications to the already implemented model were attempted. One of the simplest modifications was the change of the ReLU by the 0.2 Leaky ReLU and modifications in the order of some filters used in the residual block, however, no relevant changes were observed.

One of the first modifications was the addition of spectral normalization, described in [14] and implemented in Pytorch, to the discriminator network in order to stabilize the training and achieve better gradients. However, as observed in Figure 15, this modification did not improve the result, and on the contrary, it hides the details of the texture.

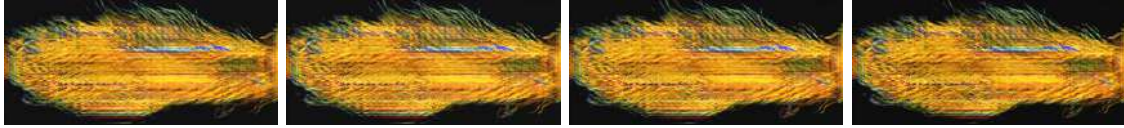


Fig. 15: Generated images WGAN-GP Resnet with spectral normalization

Other modifications were explored to solve the mode collapse. A solution described in [17] packs samples and uses them at the same time during training to help the discriminator better generalize the decision on inputs. Following this idea, a modified ResNet architecture was implemented to pack 2 samples using the Wassertein loss, this is denominated as PACGAN2. Some of the images generated for this architecture are shown in Figure 16

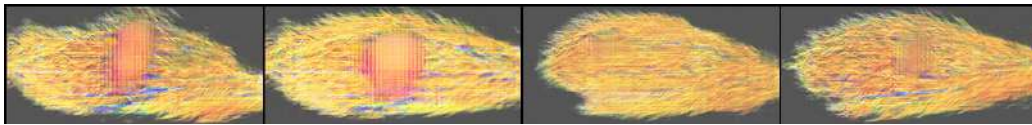


Fig. 16: Generated images PACGAN2 Wassertein-GP

Even though the mode collapse seems to be slightly alleviated, the quality of the generated images decreased significantly, and some artifacts were observed in most of images.

In addition to these solutions, a modification to the already developed network was set to incorporate Deep Local Linear Embeddings (DLLE) and Deep Isometric feature Mapping (DIsoMap) to encourage the discriminator to learn the structural information embedded in the data. The approach also involves conceptualizing the discriminator as a feature embedding mechanism, with a key goal of maximizing the entropy of distributions within this embedding. This solution was registered to combat mode collapse in [18]. The implemented code was based in the one of the authors of [18] and can be found in: <https://github.com/haozheliu-st/mee>.

The training of this model was stopped early, before the 2000 epochs, since the training seemed to be unstable with small gradients in the generator. In addition, the images did not shown signs of convergency to some shape or color. Images generated by this modified network can be observed in Figure 17. It is important to mention that possibly with some modification of the tuning parameters and larger number of epochs, this modified version could achieve better results, however, no further attempts or tuning were made in this case.

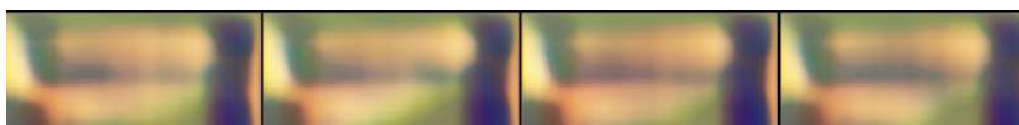


Fig. 17: Generated Images modified WGAN-GP Resnet (DLLE, DIsoMap, entropy loss)

Following the attempts to solve mode collapse reaching a perceptually good image quality, a loss modification was attempted. A modification of the loss of the WGAN was proposed in [19], this modification eliminates the constraint of limiting the Wasserstein distance to be k -Lipschitz. Instead it approximates the Wasserstein divergence, which is an approximated version of the Wasserstein metric, this leads to eliminate the gradient penalty and the gradient clipping which in practice reflects superior performance compared to the previous WGAN and WGAN-GP [19].

An implementation of this version was developed based on the code found in: https://github.com/Lornatang/WassersteinGAN_DIV-PyTorch/blob/master/wgandiv_pytorch/utils.py. Some of the generated images after training are observed in Figure 18. It is possible to observe that at first sight the mode collapse was solved while keeping an acceptable image quality. Textures and colors are captured, however, some distortion is observed in texture details.

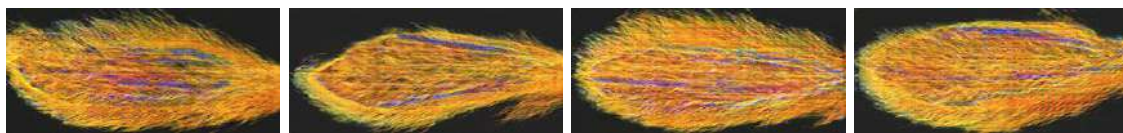


Fig. 18: Generated images WGAN-Div

In addition to this, the same ResNet network was modified to work with labels and convert the model into a conditional network. However, after several attempts of training, it seemed to exhibit a much longer time to converge and the features of the underrepresented classes were masked by the classes with the largest number of counts.

A last attempt of exploration was focused on the BigGAN implementation, however, due to difficulties with the environment and limited time, no further results were generated, even though, the network implementation is registered in the scripts.

6 Results and discussion

6.1 Network Evaluation and Selection

To evaluate the effectiveness of the pre-implemented and trained networks, a qualitative metric known as the "Fréchet inception distance" (FID) was utilized to assess the quality and diversity of the generated images. To perform this assessment, 20 images of *Sergentomyia schwetzi* specimens were generated using the models that reached the best perceivable results between the explored architectures. These generated images were then contrasted using the FID metric with the distribution of images from the original dataset, consisting of 100 representative samples.

Tab. 2: Fréchet inception distance applied to generated images using different architectures

	ResNet		Deep Convolutional(DCGAN)	AEE
	Wassertein-Div	Wassertein-GP	Wassertein-GP	MSE-Wassertein
FID	12,44	13,14	20,02	57,19

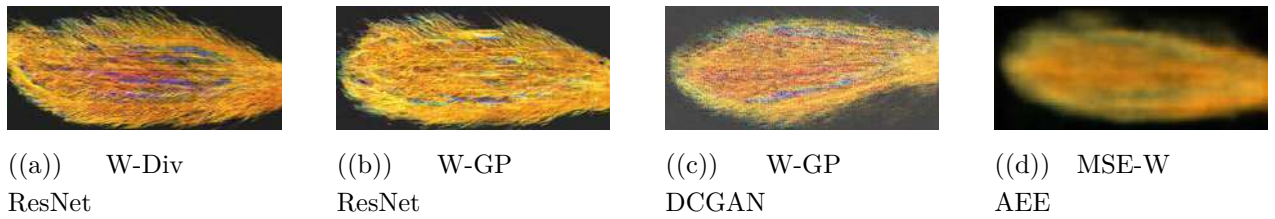


Fig. 19: Samples of Generated images

Having these evaluations, according to the FID metric, the model that reached the best result and consequently the most suited one, is the Wassertein-Divergence loss with ResNet architecture. This model also generates the samples that visually best resemble the training samples and does not exhibit a significant blurry effect and white background imprint like the AEE and DCGAN.

6.2 Image Evaluation

This selected ResNet W-Div architecture was trained indepently for the 8 Glossina specimens already specified to augment in the dataset. Each training lasted at least one day or more. In all cases, the Discriminator was trained 5 times more frequently than the generator network to stabilize the training. Also some classical augmentation techniques such as small random rotation and horizontal flip were applied to the training dataset. More details about the training parameters of this network are provided in Figure 3.

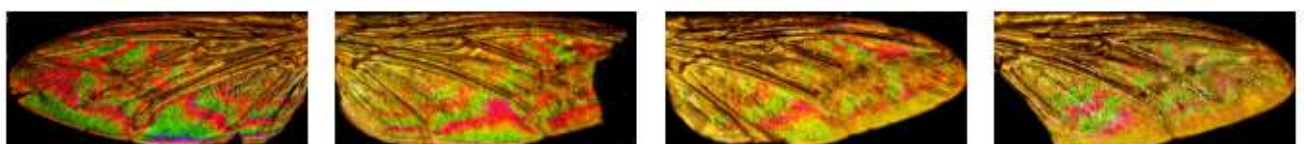
Tab. 3: Training parameter for each Glossina specimens

Category Label	Training Epochs	Training samples	Batch Size	Learning Rate	Betas
15	3310	96	16	0,0002	(0.5,0.999)
14	3994	94	16	0,0002	(0.5,0.999)
46	3060	30	8	0,0002	(0.5,0.999)
44	5000	30	8	0,0002	(0.5,0.999)
50	6000	23	8	0,0002	(0.5,0.999)
57	5856	22	8	0,0002	(0.5,0.999)
54	11999	22	8	0,0002	(0.5,0.999)
51	11240	20	4	0,0002	(0.5,0.999)

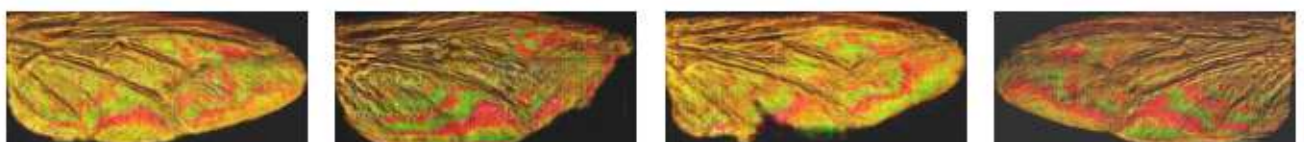
Some of the samples generated by the trained networks are shown below. As it can be observed, samples from the category 15 and 14 seem to visually resemble really well to the original ones. The colors, shapes and some textures are successfully recovered, however, some blurriness also hide some parts of the texture.

For the case of images of category 46 and 44, it seems that the blurry effect is intensified and hide significant details of the texture. The images generally recover the shape and some color, but, even though they are recovered, they seemed a bit pale in comparison to the original ones.

For categories, 50,54,51 and 57, generated images are easily distinguishable at first sight. The images present a remarked blurry effect and the texture seems plane without much detail. These significant differences are attributed mostly to the lack of training samples, since these models were trained with less than 25 pictures.

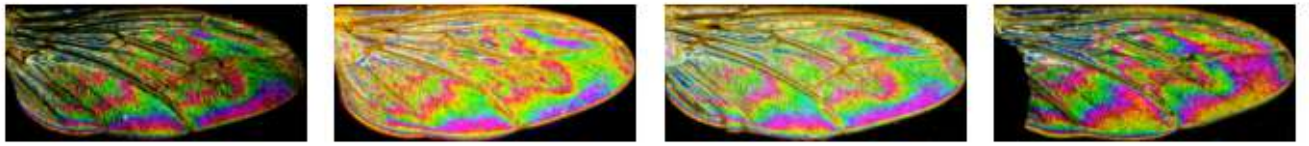


((a)) Samples from Original Dataset

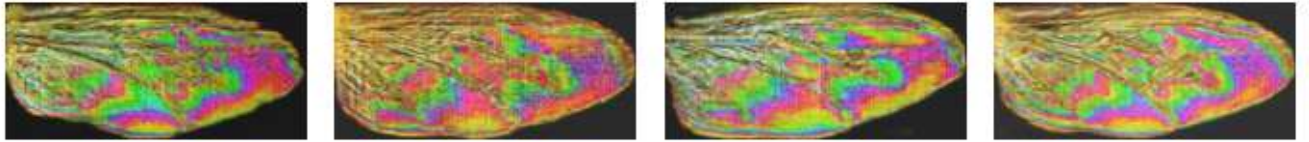


((b)) Generated Samples

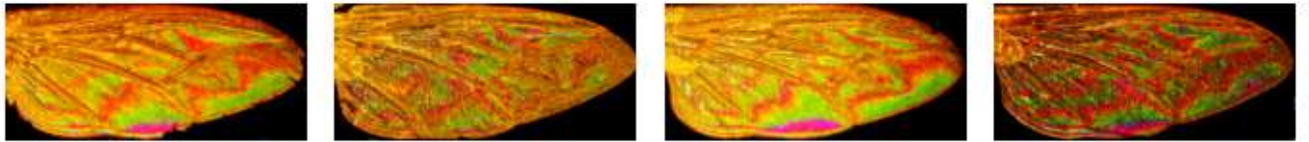
Fig. 20: Samples *Glossina fuscipes quazensis* (category 15)



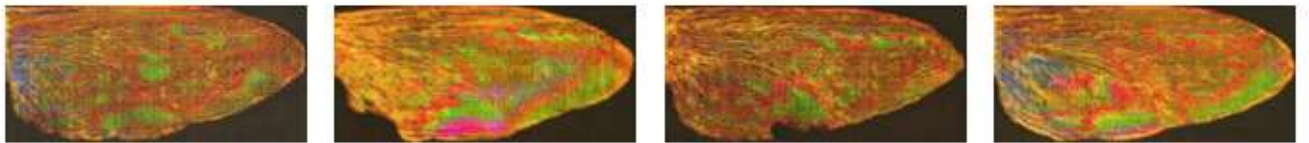
((a)) Samples from Original Dataset



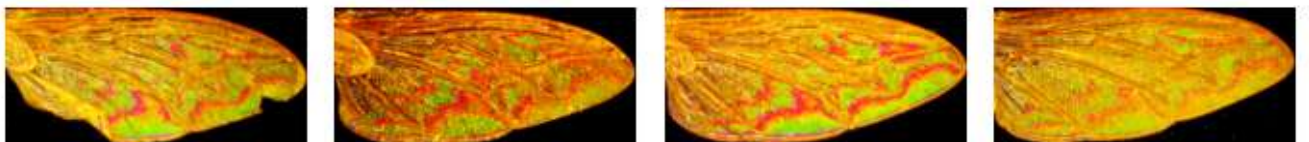
((b)) Generated Samples

Fig. 21: Samples *Glossina tachinoïdes* (label 14)

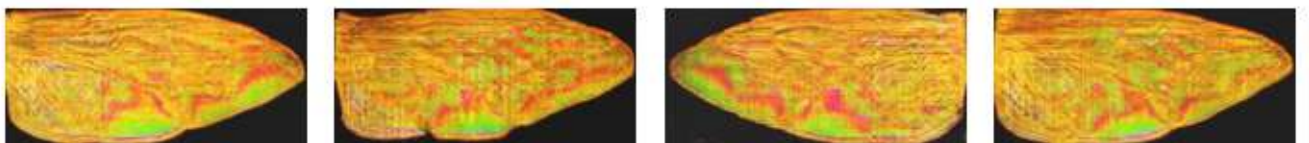
((a)) Samples from Original Dataset



((b)) Generated Samples

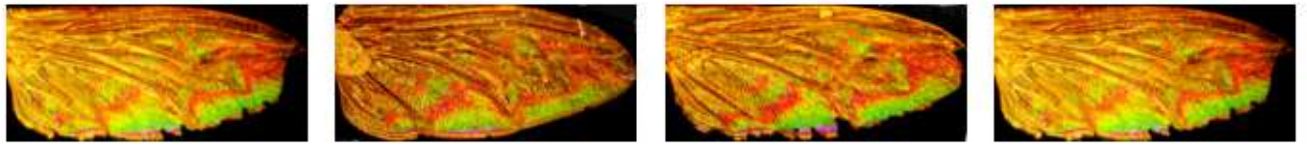
Fig. 22: Samples *Glossina morsitans submorsitans* (label 46)

((a)) Samples from Original Dataset

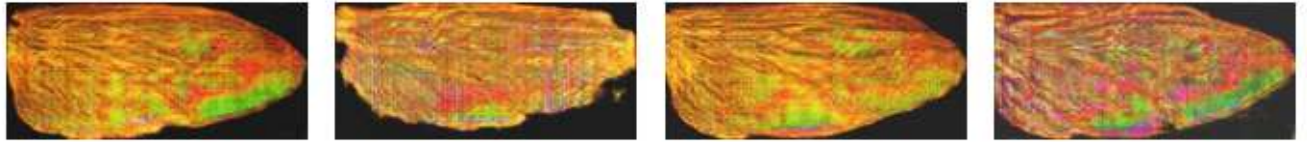


((b)) Generated Samples

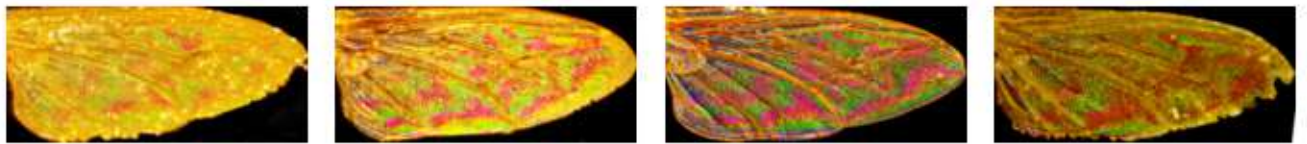
Fig. 23: Samples *Glossina longipalpis longipalpis* (label 44)



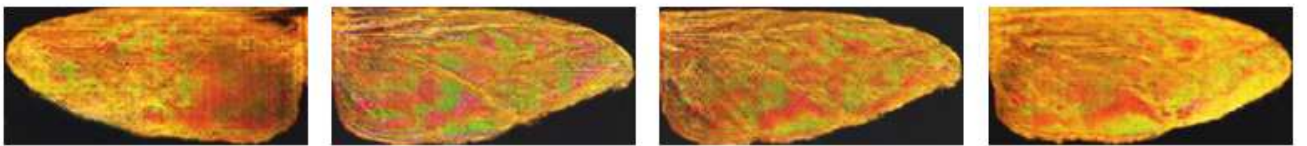
((a)) Samples from Original Dataset



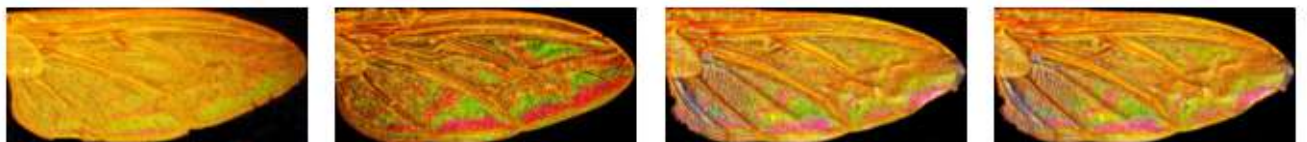
((b)) Generated Samples

Fig. 24: Samples *Glossina Caliginea* (label 50)

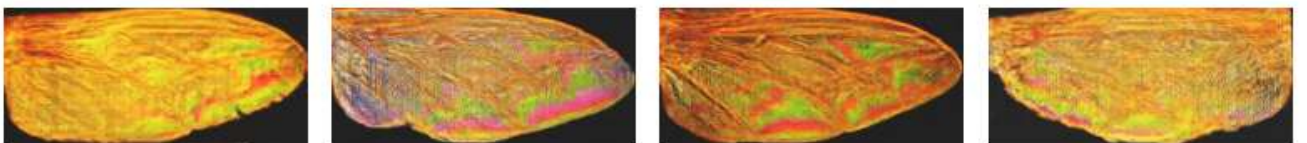
((a)) Samples from Original Dataset



((b)) Generated Samples

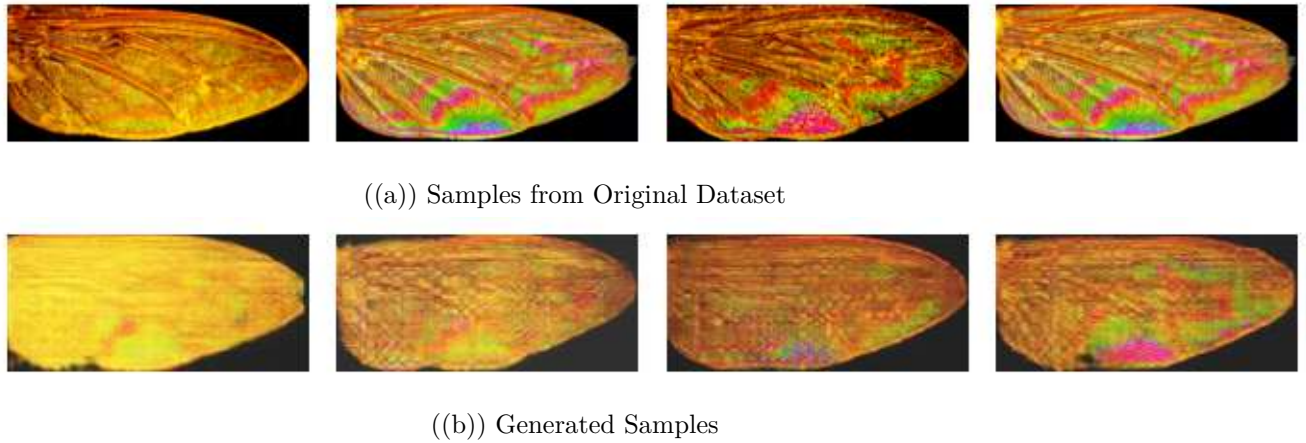
Fig. 25: Samples *Glossina Morsitans Centralis* (label 54)

((a)) Samples from Original Dataset



((b)) Generated Samples

Fig. 26: Samples *Glossina fusca fusca* (label 51)

Fig. 27: Samples *Glossina Pallicera* (label 57)

The number of images to generate was established as in Figure 4. These images were generated from a random noise prior input taken from a normal Gaussian distribution. To detect twin-generated images due to possible collapse mode in the network, the SSIM metric was calculated for all generated images in the generated space, and a threshold of 0.90 was fixed (images that surpass this threshold are considered twin or almost identical images). However, as observed in 4, the maximums SSIM for the generated images were in general much lower than this threshold, indicating that the model is performing well in terms of diversity and not twin images were generated.

Tab. 4: Comparative Metrics For original and generated WIPs

Category Label	Counts Dataset	Counts Generated	Original	Generated		SSIM Original-Generated	FID
			SSIM (min-max)	SSIM (min-max)	SSIM mean		
15	96	90	0.0381-0.8971	0.0249 - 0.4301	0.1443	0.0582	8.0388
14	94	90	0.0108-0.7952	0.0211 -0.6590	0.1575	0.0627	12.6930
46	30	60	0.0908-0.3812	0.1193 -0.4010	0.2371	0.0824	11.4025
44	30	60	0.1359-0.5118	0.0585 -0.5400	0.2033	0.0897	7.4223
50	23	50	0.0908-0.4039	0.0402-0.4578	0.1925	0.0789	8.4380
57	22	50	0.0722-0.4974	0.0694-0.5827	0.2279	0.0768	10.9234
54	22	50	0.1216-0.99	0.0480-0.6262	0.2510	0.1081	11.1398
51	20	50	0.1238-0.3969	0.0613- 0.6170	0.2197	0.1008	8.3248

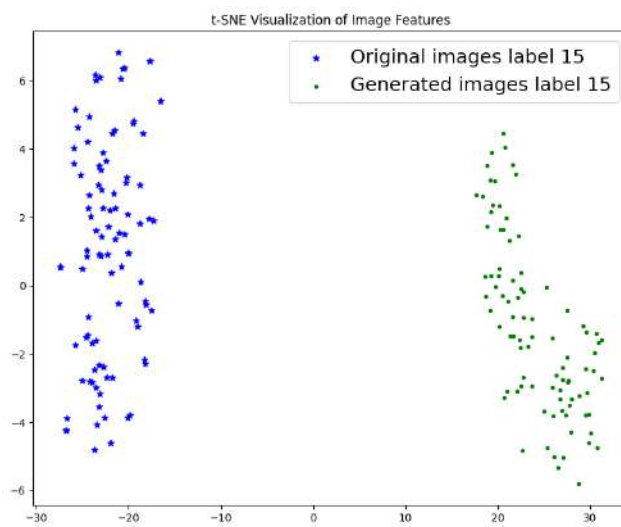
In the same way, the SSIM metric was computed for each image in the training set (original images), as well as when comparing the generated images with the entire original dataset (the provided SSIM value represents the average across the entire set). As observed in 4 according to the SSIM, only the specimens of category label 15,14,57 exhibit an average structural similarity in the range of the one presented in the original dataset. Generally,

these images present relatively low SSIM values. This result could be interpreted from 2 valid point of views. First, it indicates that the generated images are completely new representatives and show low similarity compared to the ones already presented in the dataset, which is benefit in terms of specimens diversity. Second, it could also imply that the generated samples do not adequately capture the defining features of the original dataset..

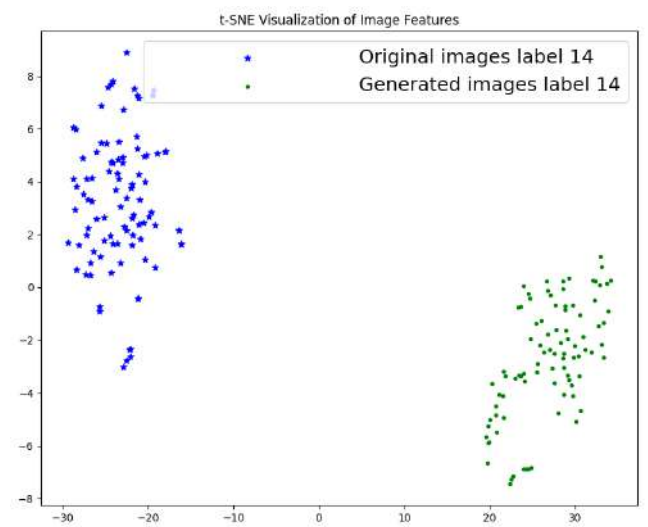
6.3 Dimensional Reduction

To observe and analyze the similarity of the features in the generated images compared to the original dataset, t-SNE dimensional reduction was employed. For this purpose, a ResNet18 network pre-trained in the ImageNet dataset was utilized as feature extractor taking the outputs before the last activation layer. In all cases 2000 iterations and perplexity equal to 30 were enough to properly visualize the reduced data.

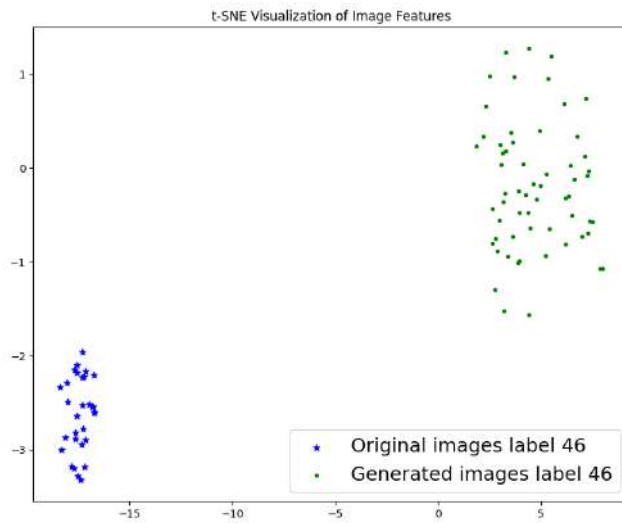
In all cases clusters were clearly defined between the generated and original data. This could be explained due to the incapacity of the current implemented model to replicate well-detailed textures. In any case, generated images are shaped so that they not completely resemble the cluster of the original dataset, while this could be beneficial in terms of bringing diversity to the dataset by specimens not seen yet in the original population, special care should be taken into account at the moment of including them as part of the training data, since it seems they do not to exhibit the characteristic features of the *Glossina* population.



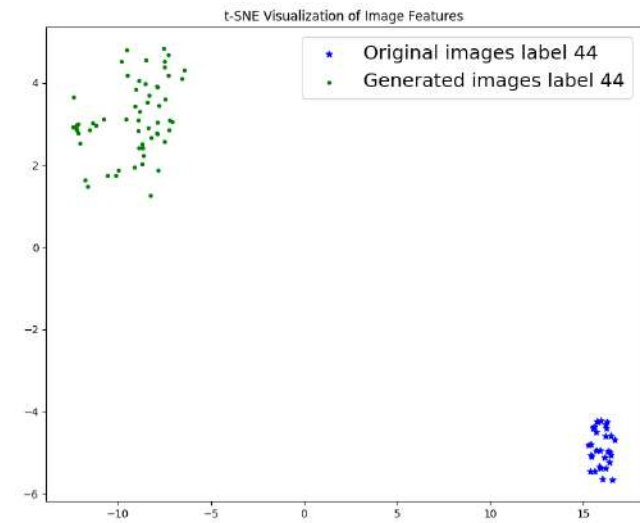
((a)) t-SNE Representation Original and generated images *Glossina fuscipes quazensis*(label 15)



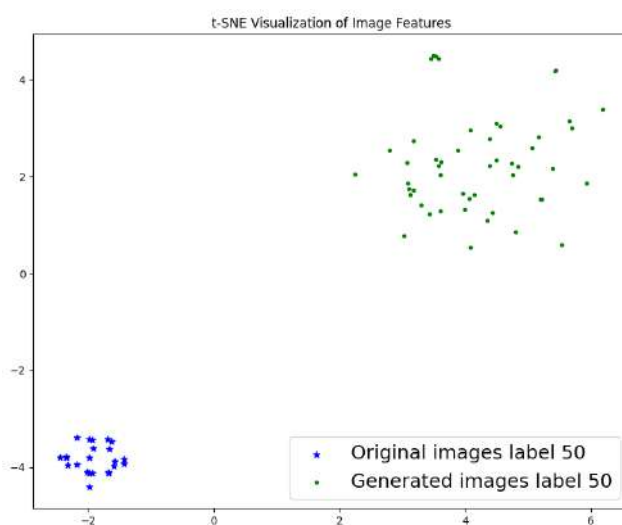
((b)) t-SNE Representation Original and generated images *Glossina tachinoïdes* (label 14)



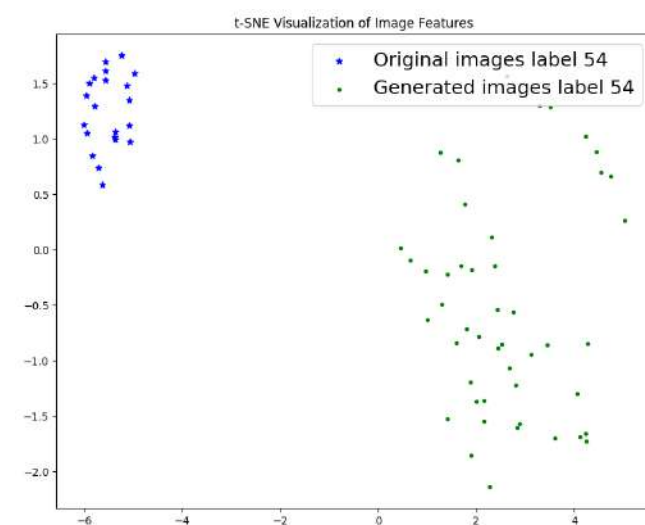
((a)) t-SNE Representation Original and generated images *Glossina morsitans submorsitans* (label 46)



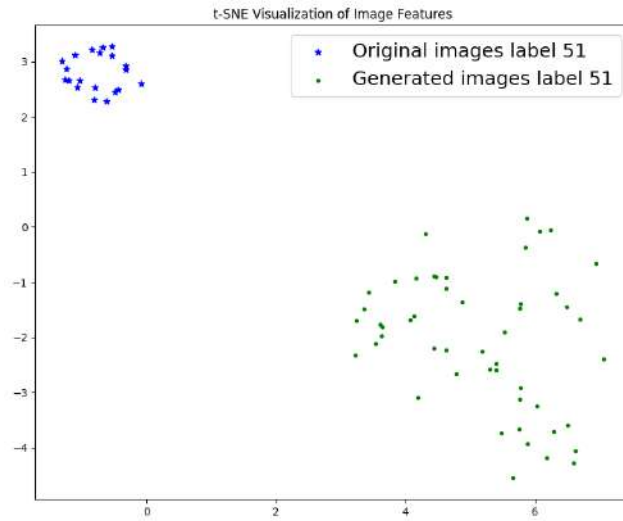
((b)) t-SNE Representation Original and generated images *Glossina longipalpis longipalpis* (label 44)



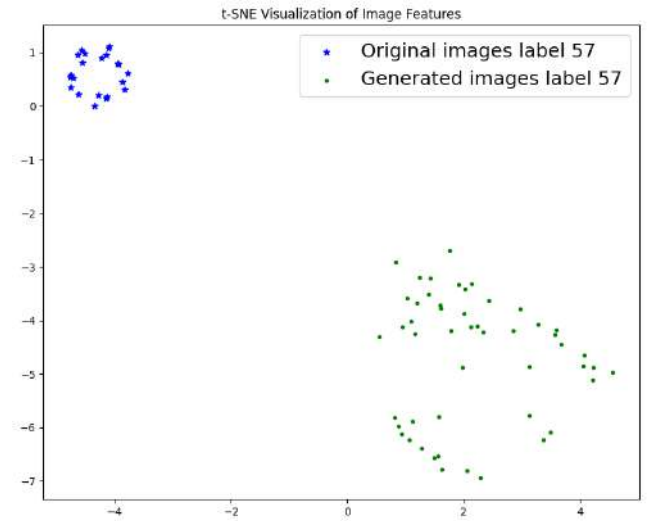
((a)) t-SNE Representation Original and generated images *Glossina Caliginea* (label 50)



((b)) t-SNE Representation Original and generated images *Glossina Morsitans Centralis* (label 54)



((a)) t-SNE Representation Original and generated images *Glossina fusca fusca* (label 51)



((b)) t-SNE Representation Original and generated images *Glossina Pallicera* (label 57)

7 Conclusion and perspectives

In conclusion, this internship project has provided valuable insights into the implementation, optimization and application of generative architectures for the generation of synthetic images and its application in the context of data augmentation. It also provided an immense experience that allowed me to engage with skills in deep learning, images processing, Python scripting, project implementation, and acquire new expertise in related tools.

The developed work successively achieved the exploration and evaluation of several GAN architectures applied to the WIPs dataset. Likewise, a suited GAN was selected to effectively augment 8 underrepresented classes. These generated images of augmentation appeared to be visually similar to the ones in the original dataset; however, some of them did not seem to obtain good quality when training with few training samples. Quantitative metrics indicates that special care should be taken into account in the case of actual use for augmentation in model training since generated images do not resemble the cluster of original features.

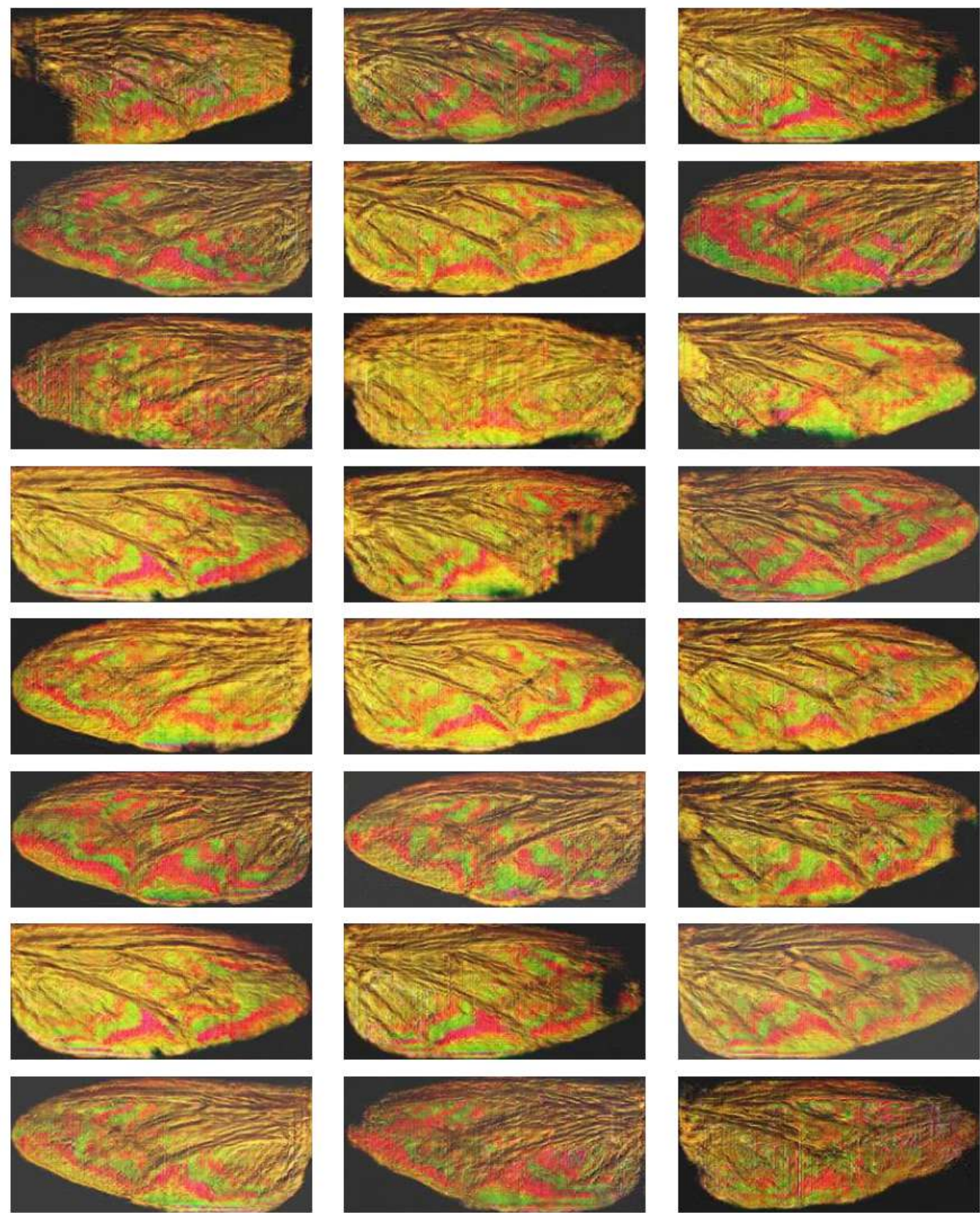
For future development, it remains to ascertain the effectiveness of the augmented images in truly enhancing the performance of the classification model. Similarly, further exploration into GAN newer models that enhance the quality of the generated images may be needed, same as the augmentation of other classes that do not belong to the *Glossina* genus.

References

- [1] A. Cannet, C. Simon-Chane, M. Akhouni *et al.*, “Wing interferential patterns (wips) and machine learning, a step toward automatized tsetse (*Glossina* spp.) identification,” *Scientific Reports*, vol. 12, p. 20086, 2022. [Online]. Available: <https://doi.org/10.1038/s41598-022-24522-w>
- [2] Etis laboratory official website. [Online]. Available: <https://www.etis-lab.fr/etis/>
- [3] Etis laboratory official website: Cell team. [Online]. Available: <https://www.etis-lab.fr/cell/>
- [4] M. Souchaud, P. Jacob, C. Simon Chane, A. Histace, O. Romain, M. Tchuenté, and S. Denis, “Mobile phones hematophagous diptera surveillance in the field using deep learning and wing interference patterns,” 10 2018.
- [5] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” 2014.
- [6] D. Saxena and J. Cao, “Generative adversarial networks (gans survey): Challenges, solutions, and future directions,” 2023.
- [7] C. Ledig, L. Theis, F. Huszar, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, “Photo-realistic single image super-resolution using a generative adversarial network,” 2017.
- [8] B. Ghojogh, A. Ghodsi, F. Karay, and M. Crowley, “Generative adversarial networks and adversarial autoencoders: Tutorial and survey,” *arXiv preprint arXiv:2111.13282*, 11 2021, accessed: Jun. 01, 2023. [Online]. Available: <http://arxiv.org/abs/2111.13282>
- [9] “Advanced courses google,” <https://developers.google.com/machine-learning/gan/training>, Google, 2023, section: GAN, Licensed under a Creative Commons Attribution 4.0 License.
- [10] “Mode collapse,” <https://machinelearning.wtf/terms/mode-collapse/>, Machine Learning Glossary, 2023, licensed under a Creative Commons Attribution 4.0 International License.
- [11] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey, “Adversarial autoencoders,” *arXiv*, May 2016.
- [12] A. Radford, L. Metz, and S. Chintala, “Unsupervised representation learning with deep convolutional generative adversarial networks,” 2016.
- [13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved training of wasserstein gans,” *arXiv preprint arXiv:1704.00028*, 12 2017, accessed: Jun. 27, 2023. [Online]. Available: <http://arxiv.org/abs/1704.00028>

- [14] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, “Spectral normalization for generative adversarial networks,” 2018.
- [15] A. Pal and A. Das, “TorchGAN: A flexible framework for GAN training and evaluation,” *Journal of Open Source Software*, vol. 6, no. 66, p. 2606, oct 2021. [Online]. Available: <https://doi.org/10.21105/joss.02606>
- [16] D. Berthelot, T. Schumm, and L. Metz, “Began: Boundary equilibrium generative adversarial networks,” *arXiv preprint arXiv:1703.10717*, 5 2017, accessed: Jun. 20, 2023. [Online]. Available: <http://arxiv.org/abs/1703.10717>
- [17] Z. Lin, A. Khetan, G. Fanti, and S. Oh, “Pacgan: The power of two samples in generative adversarial networks,” 2018.
- [18] H. Liu and et al., “Combating mode collapse in gans via manifold entropy estimation,” *arXiv preprint arXiv:2208.12055*, 4 2023, accessed: Jul. 10, 2023. [Online]. Available: <http://arxiv.org/abs/2208.12055>
- [19] J. Wu, Z. Huang, J. Thoma, D. Acharya, and L. V. Gool, “Wasserstein divergence for gans,” 2018.

8 Annex



ENSEA year 2022-2023 Fig. 32-26 Generated Images *Glossina fuscipes quazensis* (category 15)

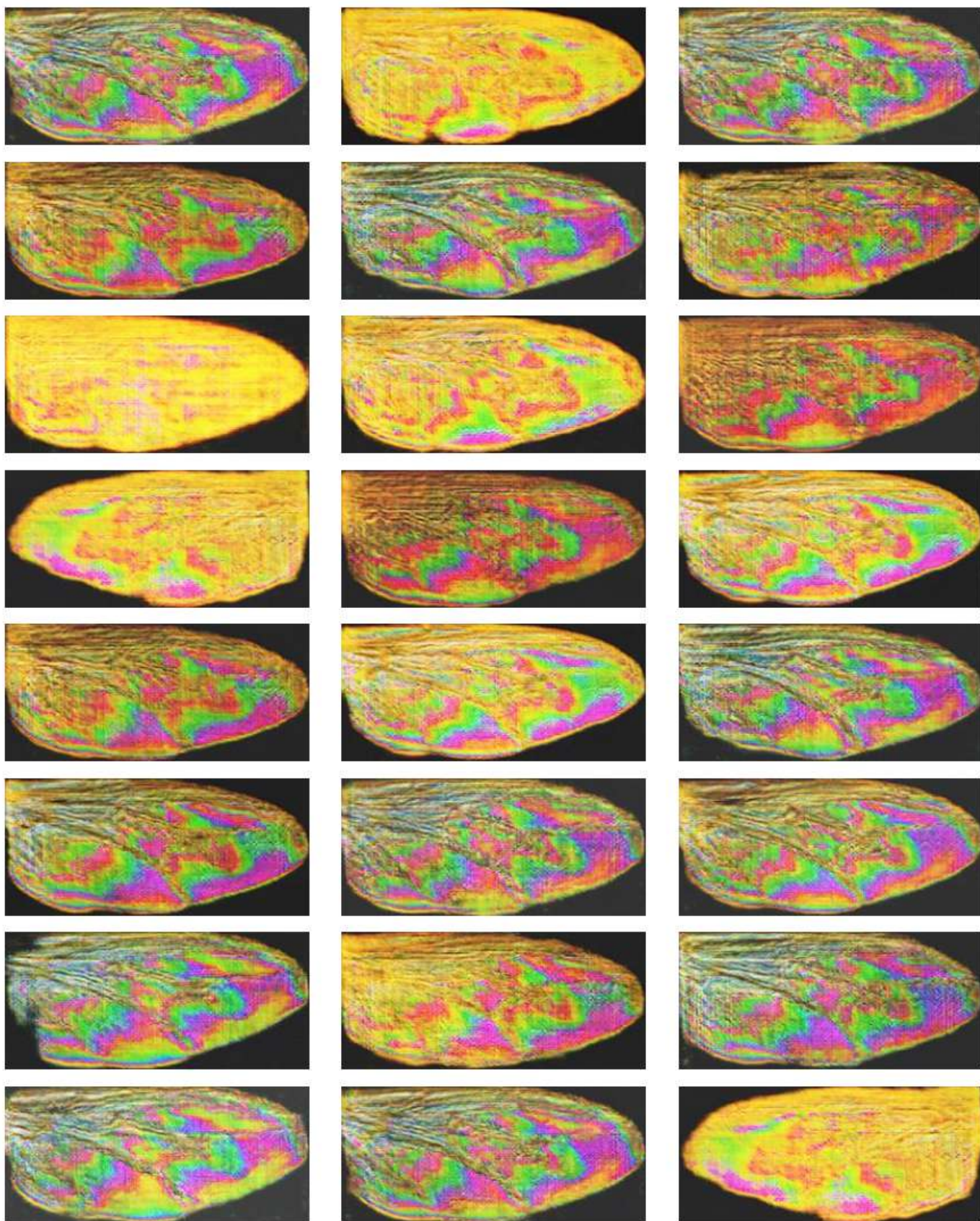


Fig. 33: Generated Images *Glossina tachinoïdes* (label 14)

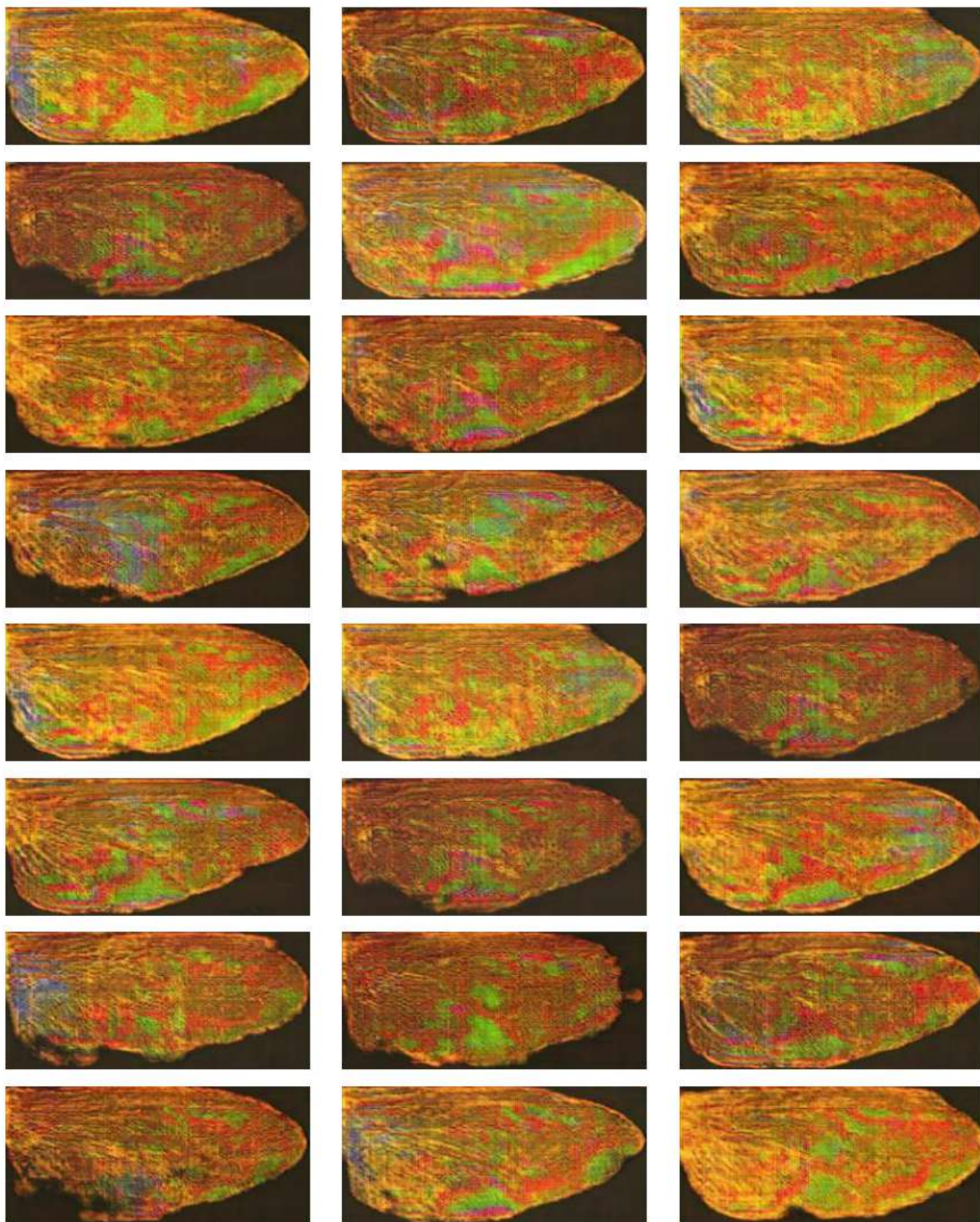


Fig. 34: Generated Images *Glossina morsitans submorsitans* (label 46)

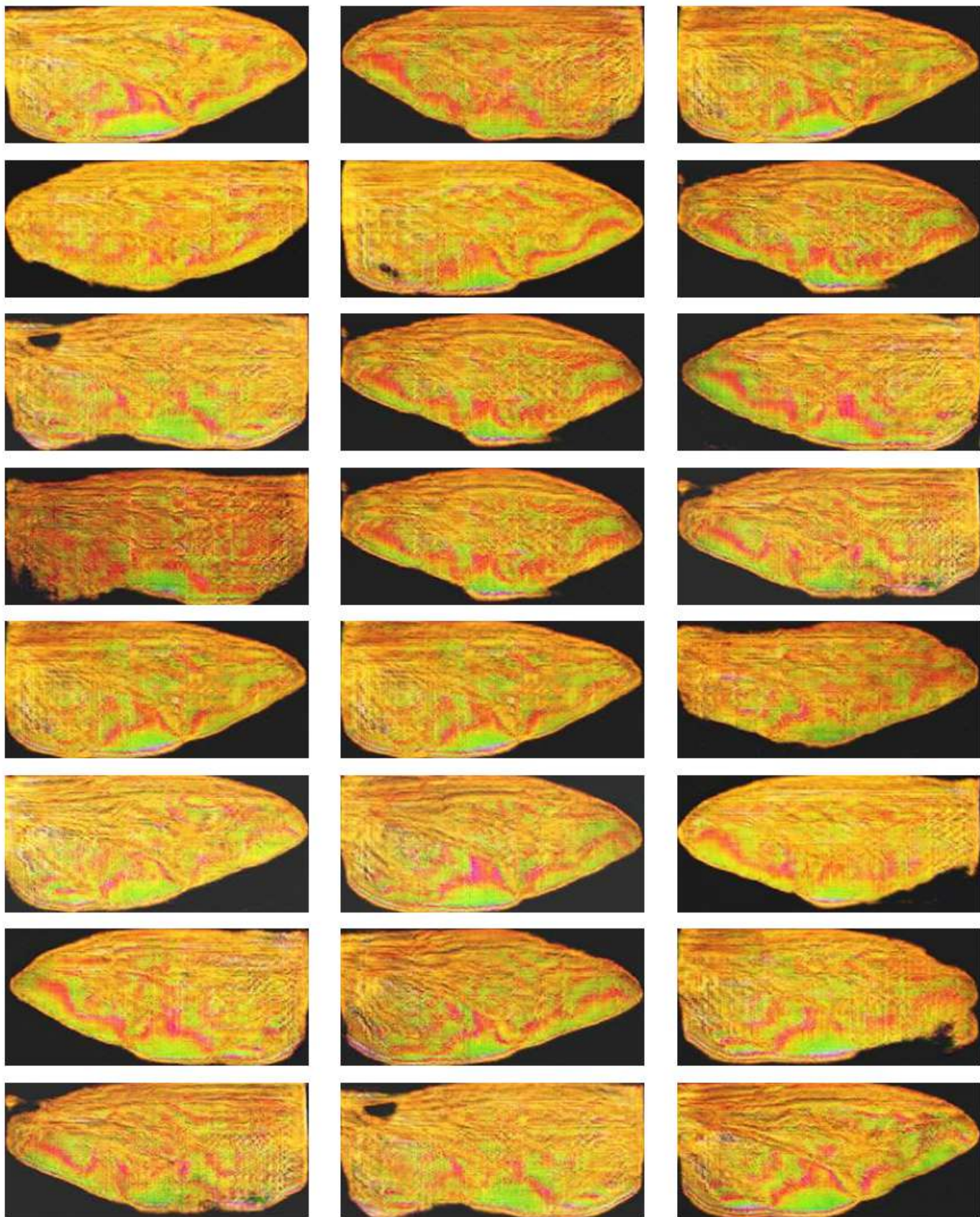


Fig. 35: Generated Images *Glossina longipalpis longipalpis* (label 44)

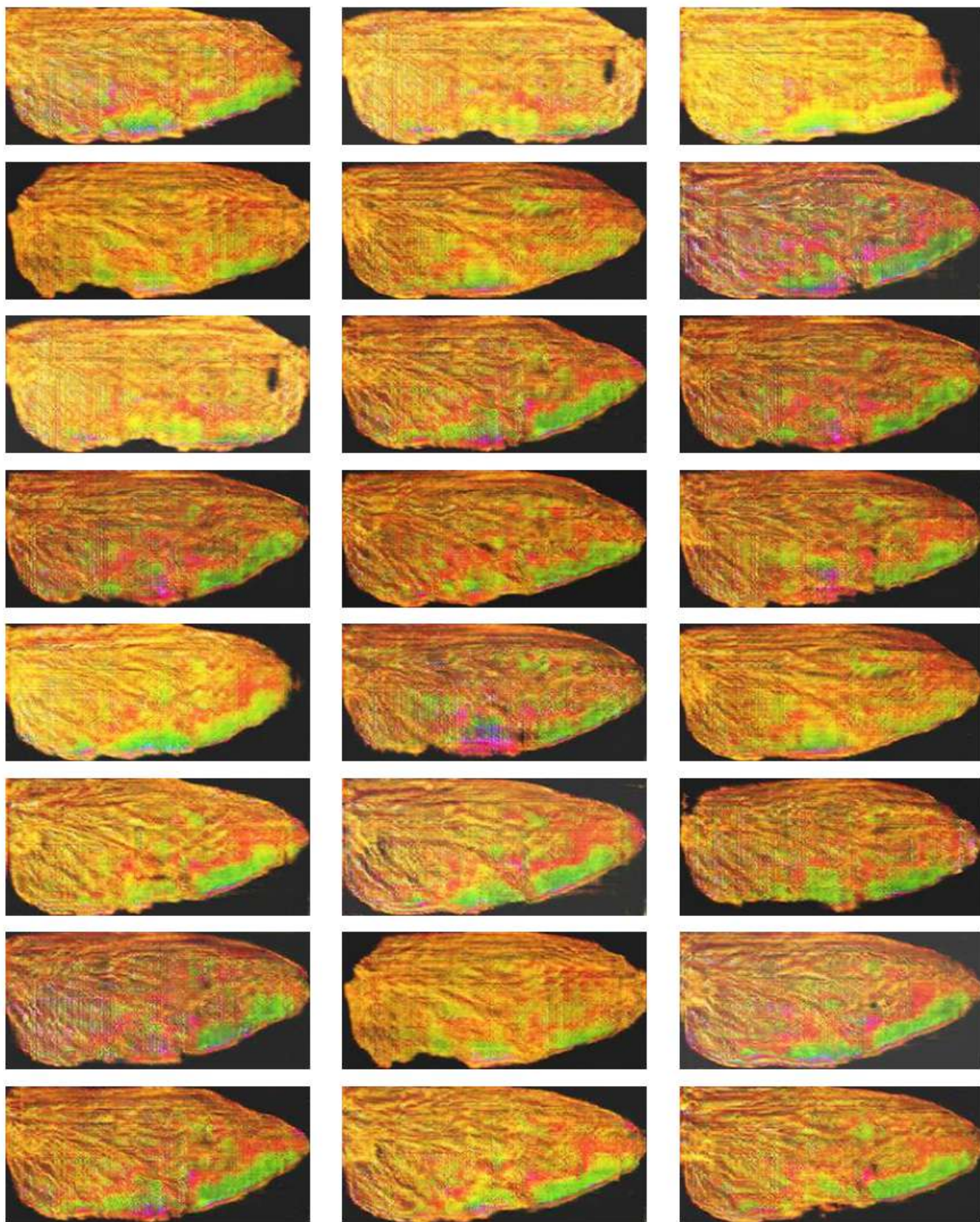


Fig. 36: Generated Images *Glossina Caliginea* (label 50)

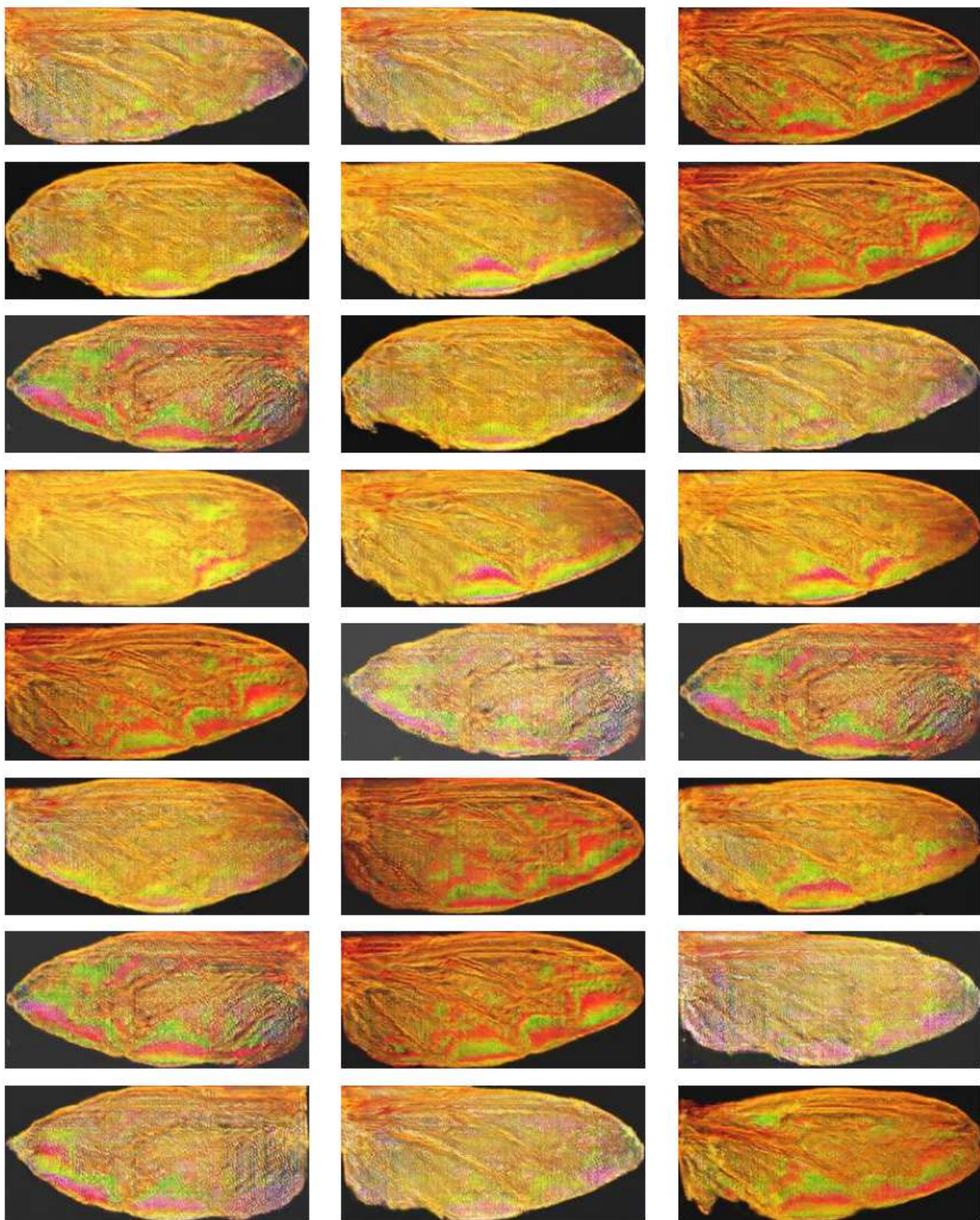


Fig. 37: Generated Images *Glossina fusca fusca* (label 51)

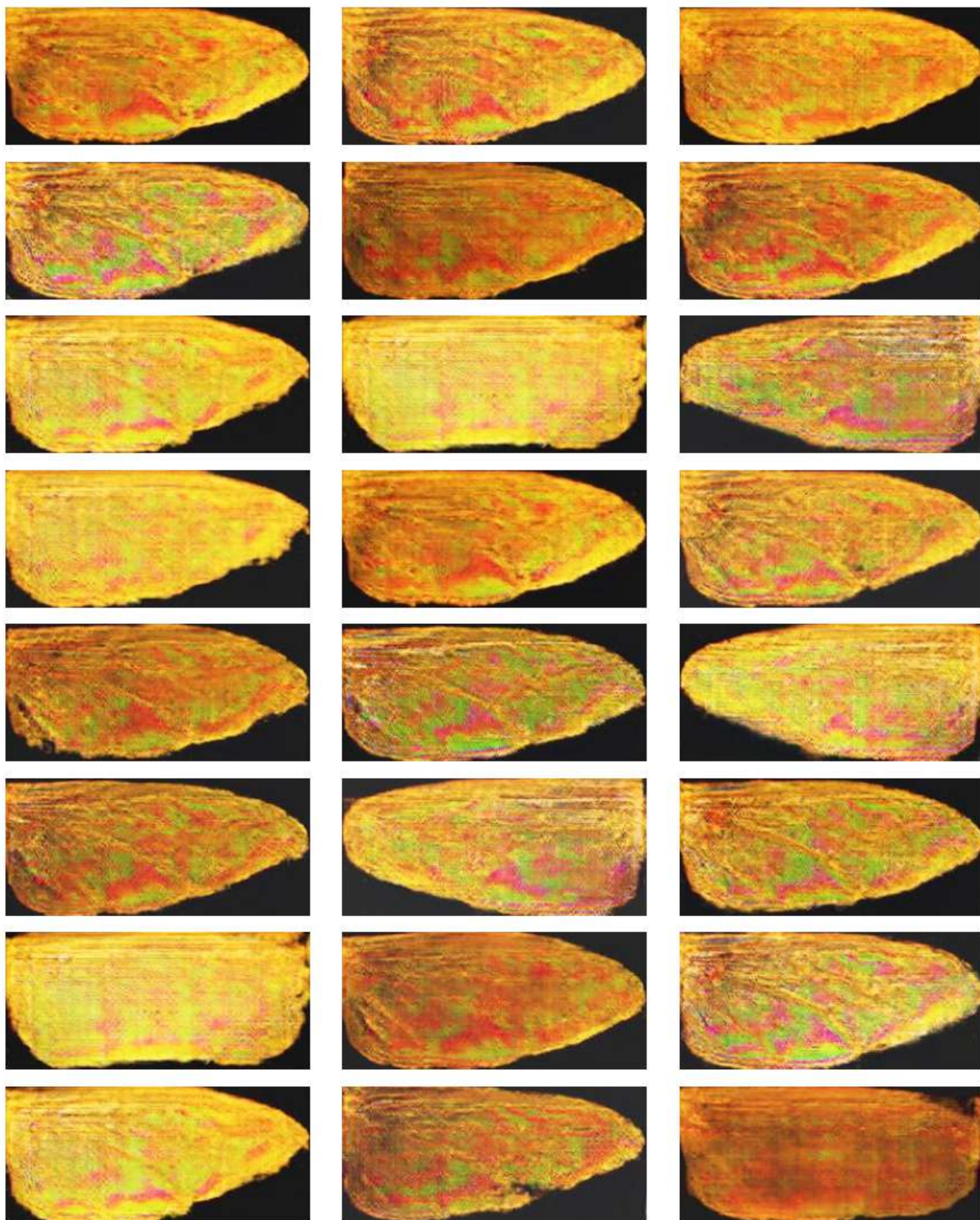


Fig. 38: Generated Images *Glossina Morsitans Centralis* (label 54)

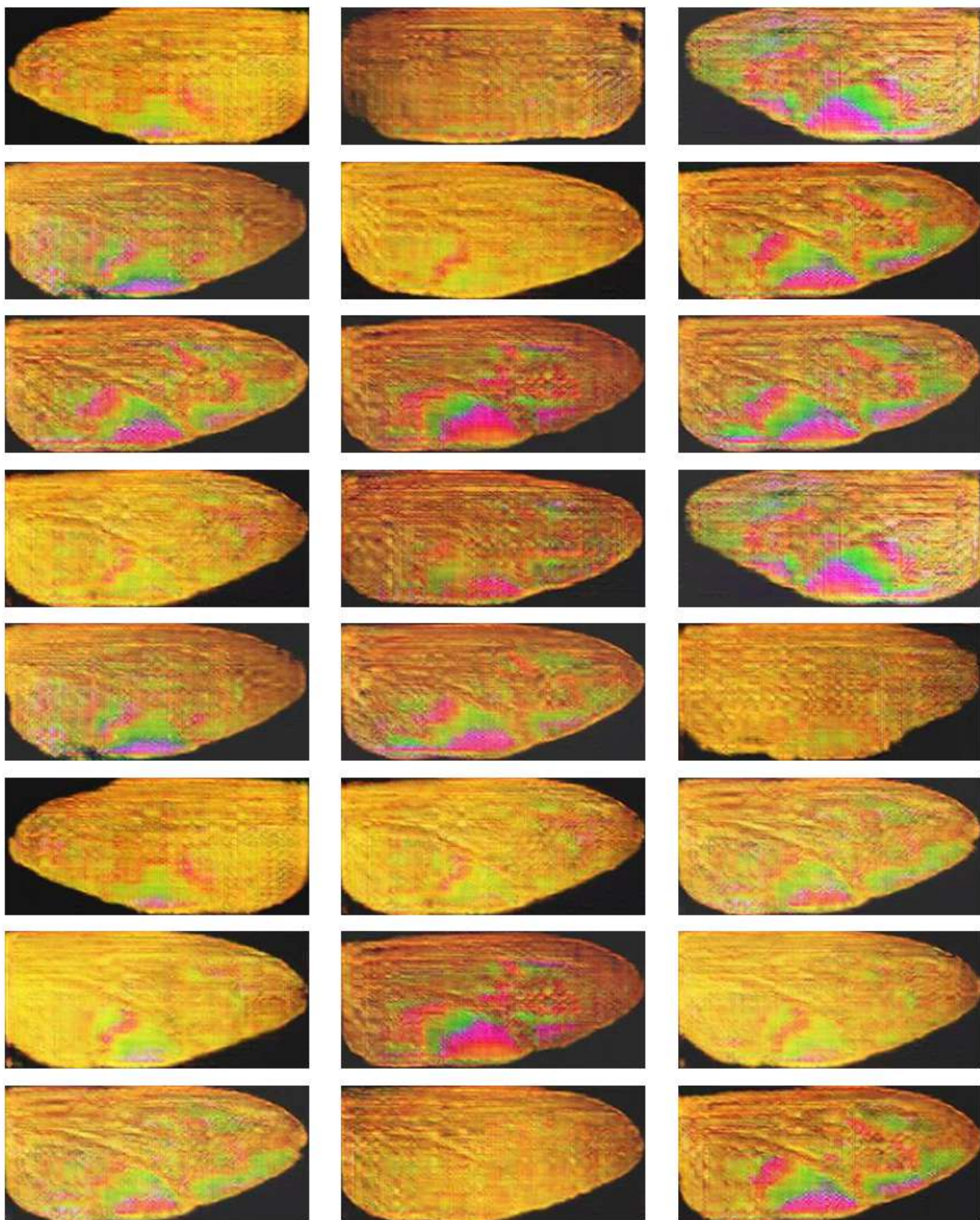


Fig. 39: Generated Images *Glossina Pallicera* (label 57)

Code: <https://github.com/jhernandezga/Deep-based-generation-of-WIPS>